API Time Machine

Project plan: a developer tool that compares API responses across versions, detects breaking changes, stores historical results, and (later) uses AI to explain the impact.

Contents

1. The project in simple terms
2. Architecture
3. Development phases
4. Software to install (Windows)
5. The smallest useful version (MVP)
6. Task 1: install and check the core tools
7. Progress checklist

1. The project in simple terms

An **API** (Application Programming Interface) is how one program asks another program for data. A weather app might send a request to a weather service, and the service sends back **JSON**, a text format for structured data:

```
{ "city": "Bengaluru", "temp": 28, "rain": false }
```

APIs change over time as developers add fields, rename things or change types. Some changes are harmless. Others quietly break every app that uses the API. The server still replies with **HTTP 200** ("OK"), so nobody notices until users complain.

API Time Machine is a "spot the difference" tool for API responses. It:

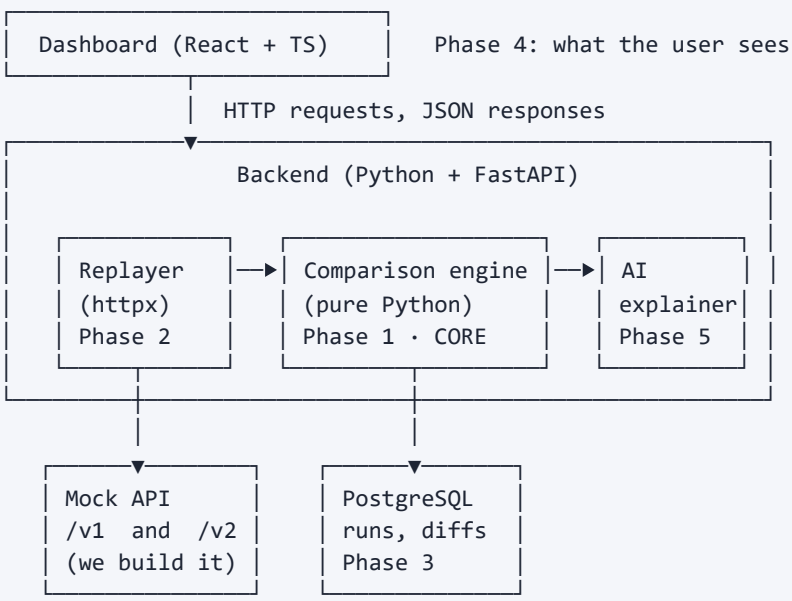
1. Takes a response from an older version and a newer one
2. Compares them field by field
3. Sorts each difference into **breaking**, **safe** or **informational**
4. Saves every comparison so you can look back at how an API changed over time
5. Later, uses AI to explain the impact in plain English

Example report:

```
BREAKING  $.price       type changed: number → string   (250 → "250")
BREAKING  $.seller.id   field removed
SAFE      $.discount    field added (number)
INFO      $.stock       value changed: 12 → 9
```

`$.price` is a **JSON path**, an address for a field. `$` means "the top of the document" and `$.seller.id` means "the `id` inside `seller`".

2. Architecture



What each part does

Part	Plain-English job	Technology
Comparison engine	Given two JSON documents, produces a list of differences with a severity for each. It has no network or database code, only logic.	Plain Python
Replayer	Sends the same request to the old and new API versions and collects both responses	`httpx` (a Python library for making HTTP requests)
Mock API	A small fake API we write with a `/v1` and a `/v2` that differ on purpose, so we never test against someone else's real system	FastAPI
Backend	A web server that ties everything together and offers endpoints like `POST /comparisons`	FastAPI
Database	Remembers past comparisons so you can see history	PostgreSQL
Dashboard	A web page to run comparisons and browse results	React + TypeScript
AI explainer	Turns "type changed: number → string" into "Mobile apps doing `price * qty` will now get string concatenation…"	An LLM API, optional and switched on with a setting

Key design decision

> The comparison engine is a **pure function**. That means the same input always gives the same output, and it doesn't touch the network, files or the database. This makes it:
>
> - **Easy to test**: just pass two JSON documents in and check the result
> - **Fast**: no waiting on networks or databases
> - **Trustworthy**: results are reproducible
>
> The AI only *explains* changes; it never *decides* whether a change is breaking. That's a strong point to make in an interview.

3. Development phases

Each phase ends with something that works and is tested. Each major phase starts only after confirmation.

Phase	What we build	Main skills you'll learn
0. Setup	Tools, Git repository, Python virtual environment, project layout	Git, terminals, environments
1. Comparison engine (core)	Compare two JSON files from the command line and print a report; the full pytest suite	Recursion, data modelling, type hints, unit testing
2. API replay	Mock API (v1/v2), httpx replayer, FastAPI endpoint that runs a comparison	HTTP, REST, async, timeouts, input validation (Pydantic), allowlists
3. History	Save runs and changes in PostgreSQL; endpoints to list and view past runs	SQL, schema design, ORM (SQLAlchemy), migrations, environment variables
4. Dashboard	React UI: run a comparison, see the diff colour-coded, browse history	Components, state, fetching data, TypeScript types
5. AI explanations	Optional explanations layered on top of the deterministic results	Prompt design, API keys handled safely, fallbacks
6. Docker + GitHub	One-command startup with Docker Compose; GitHub Actions running tests on every push	Containers, CI/CD
7. Polish	README, architecture diagram, screenshots, sample data, demo script	Communicating your work

Every phase uses Git commits, type hints, error handling and tests. Those aren't saved for the end.

4. Software to install (Windows)

Install only the first group now. Each later item gets installed at the start of its phase.

Install now (Phase 0–1)

Software	Why	Where
Git	Saves snapshots ("commits") of your code so you can undo mistakes and show your history	git-scm.com → Download for Windows
Python 3.13 or 3.14	Runs the backend and comparison engine	python.org → Downloads
VS Code	Code editor	code.visualstudio.com
VS Code extension: Python (by Microsoft)	Code completion, error highlighting, debugging	Inside VS Code → Extensions panel
GitHub account	Hosts your code publicly for recruiters (you can create it anytime before Phase 6)	github.com

Install later

- **PostgreSQL**: Phase 3
- **Node.js (LTS version)**: Phase 4, needed for React
- **Docker Desktop**: Phase 6

5. The smallest useful version (MVP)

MVP (Minimum Viable Product) means the smallest thing that already solves the real problem.

Our MVP is a command-line tool that compares two JSON files:

```
python -m api_time_machine compare old.json new.json
```

It prints a report like the one in section 1. It should handle:

- Fields **added** and **removed**
- **Type changes** (number → string, object → list, value → `null`)
- **Value changes** with the same type
- **Nested objects** (e.g. `$.seller.address.city`)
- **Lists** (e.g. `$.items[0].price`)

It comes with a **pytest test suite** of roughly 10–15 cases, one per kind of change.

Why start here

- It's the heart of the project. Everything else (web server, database, UI, AI) just feeds data into it or displays its output.

- It needs no network, database or frontend, so there's very little that can go wrong apart from your logic.
- It's the part interviewers will ask about most ("How does your diff algorithm handle arrays?"), so you'll understand it deeply.

6. Task 1: install and check the core tools

Step 1: install Git

- Download from git-scm.com and run the installer.
- Accept the defaults, with one exception: when asked for the default editor, choose **"Use Visual Studio Code as Git's default editor"**.

Step 2: install Python

- Download from python.org.
- If the installer shows a checkbox **"Add python.exe to PATH"**, tick it.
- **PATH** is the list of folders Windows searches when you type a command. Without it, typing `python` gives "not recognized".

Step 3: install VS Code

Open it, go to Extensions (Ctrl+Shift+X), and install **Python** by Microsoft.

Step 4: verify

Close any open terminals. Open a new **PowerShell** window (Start menu → type "PowerShell") and run each of these commands one at a time:

```
git --version
python --version
python -m pip --version
code --version
```

Expected output (your numbers may differ slightly):

```
git version 2.4x.x.windows.1
Python 3.13.x       (or 3.14.x)
pip 2x.x from C:\...\Python313\...
1.9x.x  (followed by two more lines)
```

Step 5: tell Git who you are

This name and email get stamped on every commit you make. Use the email you'll use for GitHub.

```
git config --global user.name "Your Full Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

The last command makes new projects use `main` as the name of their primary branch, which is the modern standard.

Debugging lessons: errors you might see

`'git' is not recognized as the name of a cmdlet...`

- **Cause:** Windows can't find the program on PATH. This usually happens because the terminal was opened *before* the install finished.
- **Fix:** close the terminal and open a new one.
- **Lesson:** a terminal reads PATH only when it starts.

Typing `python` opens the Microsoft Store, or prints nothing

- **Cause:** Windows has a built-in "app execution alias" shortcut that hijacks the `python` command.
- **Fix:** Settings → Apps → Advanced app settings → App execution aliases → turn **off** the two "python" entries. Then reopen PowerShell.
- **Workaround:** `py --version` often works even when `python` doesn't.

When you're done

Share the output of the four `--version` commands, plus any errors. Next comes the rest of Phase 0: creating the project folder, starting the Git repository and setting up a Python virtual environment.

7. Progress checklist

☐ **Phase 0:** Setup
  ☐ Task 1: Install Git, Python, VS Code and verify
  ☐ Create project folder and Git repository
  ☐ Create Python virtual environment

☐ **Phase 1:** Comparison engine (MVP)
☐ **Phase 2:** API replay + mock API
☐ **Phase 3:** Historical results (PostgreSQL)
☐ **Phase 4:** Dashboard (React + TypeScript)
☐ **Phase 5:** AI explanations
☐ **Phase 6:** Docker + GitHub Actions
☐ **Phase 7:** README, diagram, screenshots, demo